Introduction to Automation & Selenium

- **What is Automation?**

Automation means using tools or software to perform tasks without manual intervention.

In software testing, automation is mainly used to replicate repetitive human actions so that tasks can be done faster and more consistently.

- **Why Test Automation?**

Test automation is useful because it offers several advantages:

Speed: Automated tests run much faster than manual testing.
Efficiency: It saves time and effort in the long run, especially for repeated test execution.
Reliability: It reduces human error in repetitive tasks.
Reusability: Automation scripts can be reused multiple times across different versions and environments.
Repeatability: It ensures that tests are executed in the same way every time.
Regression Testing: It is very useful for running large sets of tests after code changes to make sure existing features are not broken.
Coverage: Automation makes it possible to test more scenarios and data combinations than would be practical with manual testing.

- **Disadvantages of Automation**

Although automation is powerful, it also has some limitations:

Initial Cost: Setting up automation frameworks and writing the first set of scripts requires time, effort, and resources.

Maintenance: Automation scripts must be updated whenever the application changes.

Skilled Resources: It requires programming knowledge and technical skills.

Not a Complete Replacement for Manual Testing:
Automation cannot handle everything.
It is not suitable for:

- exploratory testing
- usability testing
- CAPTCHA validation
- scenarios that need human judgment

So, automation cannot completely replace manual testing.

- **What is Selenium?**

Selenium is a suite of open-source tools used to automate web browsers.
It is mainly used for functional testing of web applications.
Selenium interacts with web elements such as:

- buttons
- text fields
- links

It works in a way similar to how a real user interacts with a website.

- **Selenium 4 Key Changes**

Selenium 4 introduced several important improvements that make browser automation more stable, standard, and easier to use.

1. W3C WebDriver Protocol Compliance

Selenium 4 follows the **W3C WebDriver standard**.
This means communication between WebDriver and browsers is now standardized.

Benefits:

- more stable test execution
- better consistency across different browsers
- fewer browser-specific issues
- improved compatibility between Selenium and browser drivers

2. Relative Locators

Selenium 4 introduced **Relative Locators**, which provide a more intuitive way to locate elements based on their position compared to other elements.
Examples include locating an element:

- above another element
- below another element
- near another element
- to the left of another element
- to the right of another element

Benefit:

This makes element identification easier when normal locators are difficult to use.

3. Improved Window and Tab Management

Selenium 4 made handling multiple browser windows and tabs simpler.
It provides easier commands for:

- opening new windows or tabs
- switching between windows or tabs
- managing multiple browser sessions more effectively

Benefit:

This improves readability of test scripts and makes multi-window automation easier to handle.

Essential HTML for Testers

- **What is HTML?**

HTML (HyperText Markup Language) is the standard language used to create web pages.
It defines the **structure** and **content** of a webpage using tags.
Browsers read and interpret HTML to display the visual page to users.

- **HTML Tags**

HTML elements are written using **tags**, which are enclosed in angle brackets: < >
Most tags come in pairs:

- **Opening tag** → <tag>
- **Closing tag** → </tag>

Common HTML Tags for Testers
<html>: The root element of an HTML document.
<head>: Contains meta-information about the page, such as:

- title
- links to CSS
- links to JavaScript files

<title>: Defines the text shown in the browser tab or title bar.
<body>: Contains all the visible content of the webpage.
** :** A block-level container used to divide sections of a page. It is often used for layout purposes.
: An inline container, usually used for styling or grouping small parts of text.
<a>: Anchor tag used to create hyperlinks.
<input>: Used for input fields such as:

- text box
- password field
- checkbox
- radio button
- button
- file upload

<button>: Defines a clickable button.
<select>: Represents a dropdown list container.
<option>: Represents an item inside a <select> dropdown.
<img>: Used to display images.
Table Tags
Used for creating tables:

- <table> → table container
- <tr> → table row
- <th> → table header cell
- <td> → table data cell

List Tags

Used for lists:

- <ul> → unordered list
- <ol> → ordered list
- <li> → list item

<iframe>

Embeds another HTML page inside the current page.

❖ **HTML Attributes**

Attributes provide extra information about an element.

They are written inside the opening tag, usually in the format:

name="value"

Common Attributes Used in Selenium

id

- Should be unique on the page
- Usually the best locator when available and stable

name

- Commonly used in form elements
- Does not have to be unique

class

- Used for CSS styling
- An element can have multiple classes
- Often used as a locator, but may match many elements

href

- Used with <a> tags
- Specifies the destination URL of the link

src

- Used with <img> tags
- Specifies the image source URL

value

- Represents the default or current value of an input element
- Also used for select and radio button values

type

- Mostly used with <input>
- Defines the input type, such as:
    - text
    - password
    - radio
    - checkbox
    - submit
    - file

title

- Often shown as a tooltip when hovering over the element

Other Useful Attributes

- style
- placeholder

➢ DOM (Document Object Model)

The **DOM** is a tree-like structure created by the browser from the HTML page.
Selenium interacts with this DOM, not with the visual page directly.
Understanding DOM relationships is important for advanced locators such as XPath and CSS selectors.

Important DOM Relationships

- **Parent**
- **Child**
- **Sibling**

These relationships help in locating elements more accurately.

➢ Inspecting Elements

Why Inspect Elements?

Inspecting elements helps you:

- understand the HTML structure of the page
- find important attributes like id, name, and class
- identify the correct locators for Selenium scripts

How to Inspect Elements?

You can inspect elements using browser developer tools.

Chrome

- Right-click on the element → **Inspect**
- Or press **F12**
- Crtl + Shift + I

Firefox

- Right-click on the element → **Inspect Element**
- Or press **F12**

Edge

- Right-click on the element → **Inspect**
- Or press **F12**

Key Features in Developer Tools

Elements / Inspector Tab

This tab shows the live HTML DOM of the webpage.
You can:

- expand or collapse HTML nodes

- view nested elements
- hover over HTML code to highlight the related element on the page
- hover over an element on the page to locate it in the DOM

Selector Tool

Usually shown as a cursor inside a box.

You can:

1. click the selector tool
2. click any element on the page
3. jump directly to that element's HTML code in the Elements tab

Search in Elements Tab

Use **Ctrl + F** inside the Elements tab.

This allows you to search using:

- plain text
- CSS selectors
- XPath

It is very useful for testing locators directly in the browser.

❖ **Basic WebDriver Commands**

driver.get(url)

Opens the specified URL in the browser.

driver.maximize_window()

Maximizes the browser window

Why we use it:

1. **Consistency in Tests**
    - Some elements may not be visible or clickable if the window is small.
    - Maximizing ensures elements are displayed as expected.
2. **Responsive Design Handling**
    - Websites may render differently on small windows.
    - Maximizing helps avoid layout issues that can break automated tests.
3. **Better Visibility**
    - Helps testers see the entire page while debugging scripts.
    - Useful for screenshots in automated tests.

driver.minimize_window()

Minimizes the browser window to the taskbar.

Browser is still running, but not visible on the screen.

It is optional and used for background execution or resource management.

driver.title

Returns the title of the current page.

driver.current_url

Returns the current page URL.

driver.name
Used to get the **name of the browser driver** being used in Selenium.
driver.quit()
Closes the entire browser session and ends the WebDriver session.
When to use:

- Use quit() when your **test script is finished** and you want to **clean up all browser windows and WebDriver resources**.
- This is the **recommended method at the end of every Selenium script** to avoid leaving browser processes running in the background.

driver.close()
Closes only the current browser window or tab.
When to use:

- Use close() when you have **multiple windows/tabs open** and you want to **close only one** of them.
- Example: After opening a popup window and finishing actions there, you can close just the popup without affecting the main window.

Navigation Commands
driver.back()
Works like clicking the browser's **Back** button.
driver.forward()
Works like clicking the browser's **Forward** button.
driver.refresh()
Reloads the current webpage.

- ❖ **Finding Elements in Selenium**

Finding elements is the core part of Selenium automation.
Goal
To uniquely identify the exact element or elements you want to interact with.
Strategy
Use the **By** class with different locator strategies.
Import Statement
from selenium.webdriver.common.by import By

- **Methods for Finding Elements**

driver.find_element(By.LOCATOR_STRATEGY, "value")
Finds the **first matching element**.
If no matching element is found, Selenium throws a: NoSuchElementException
driver.find_elements(By.LOCATOR_STRATEGY, "value")
Finds **all matching elements** and returns them as a list of WebElements.
If no elements are found, it returns:

- an empty list

- **Example Code:**

Opening Chrome Browser:

```
from selenium import webdriver
driver = webdriver.Chrome()
driver.get("https://www.google.com")
print("Browser opened:", driver.name)
driver.quit()
```

from selenium import webdriver
This imports the webdriver module from Selenium.
webdriver is used to control browsers like Chrome, Edge, and Firefox through Python.
driver = webdriver.Chrome()
This starts a new **Chrome browser session**.
Selenium launches Chrome and stores its control inside the variable driver.
driver.get("https://www.google.com")
This opens the given URL in the Chrome browser.
Here, it opens Google.
print("Browser opened:", driver.name)
This prints the name of the browser driver being used.
driver.quit()
This closes the entire browser window and ends the Selenium session properly.

Opening Edge Browser:

```
from selenium import webdriver
driver = webdriver.Edge()
driver.get("https://www.google.com")
print("Browser opened:", driver.name)
driver.quit()
```

Opening Firefox Browser:

```
from selenium import webdriver
driver = webdriver.Firefox()
driver.get("https://www.google.com")
print("Browser opened:", driver.name)
driver.quit()
```

Using Chrome Options in Selenium

ChromeOptions allows us to **customize the behavior of Chrome browser** before launching it.

```
from selenium import webdriver
from time import sleep

opts = webdriver.ChromeOptions()

opts.add_experimental_option('detach',True)
opts.add_argument('--headless')
opts.add_argument('--incognito')
opts.add_argument('--disable_notifications')

driver = webdriver.Chrome(options = opts)

driver.get('https://cornerhouseicecreams.com/')
driver.maximize_window()

print(f'current url is > {driver.current_url}')
print(f'title of the page is > {driver.title}')

sleep(5)
driver.quit()
```

OUTPUT:

```
current url is > https://cornerhouseicecreams.com/
title of the page is > Corner House - Bengaluru’s Favorite Ice Cream & Desserts
```

Code Example with Explanation

```
from selenium import webdriver
from time import sleep
```

- **Imports required modules**:
  - webdriver → for Selenium browser automation.
  - sleep → to pause execution for a few seconds (used for demo purposes).

```
opts = webdriver.ChromeOptions()
```

- **Creates a ChromeOptions object** called opts.
- We will use this object to set Chrome-specific preferences before launching the browser.

```
opts.add_experimental_option('detach', True)
```

- Keeps the browser **open after the script finishes**.
- Useful when you want to see the final state of the browser after automation completes.
- Without this, Chrome would automatically close after driver.quit().

```
opts.add_argument('--headless')
```

- Runs Chrome in **headless mode** (no UI).
- Useful for running tests on servers or CI/CD pipelines where you don't need to see the browser.

```
opts.add_argument('--incognito')
```

- Opens Chrome in **Incognito/Private mode**.
- Ensures a fresh session without cache, cookies, or browsing history.

```
opts.add_argument('--disable_notifications')
```

- **Disables pop-up notifications** from websites.
- Helps avoid interruption in automated tests.

```
driver = webdriver.Chrome(options=opts)
```

- Launches Chrome with all the options set in opts.
- Stores the browser instance in driver.

```
driver.get('https://cornerhouseicecreams.com/')
```

- Opens the given URL in Chrome.

```
driver.maximize_window()
```

- Maximizes the browser window for **consistent element visibility**.

---

sleep(5)

- Pauses the script for **5 seconds**.
- Allows the page to load fully or lets us observe the browser if detach=True.

---

driver.quit()

- Closes all browser windows and ends the WebDriver session.

Note:

- Even though detach=True, quit() will still close the browser. To actually leave the browser open, you would **omit driver.quit()**.
- Always pass options=opts to webdriver.Chrome() to apply the settings.

FOR EDGE:

```
opts = webdriver.EdgeOptions()

opts.add_experimental_option('detach', True)
opts.add_argument('--headless')
opts.add_argument('--inprivate') # Edge private mode
opts.add_argument('--disable-notifications')

driver = webdriver.Edge(options=opts)
```

FOR FIREFOX:

```
opts = webdriver.FirefoxOptions()

opts.add_argument('--headless')
opts.add_argument('-private')   # Firefox private mode

driver = webdriver.Firefox(options=opts)
```

Note: By default Firefox stays open so no need to use ‘detach’

Basic Locator Strategies in Selenium

Locator strategies are used to identify web elements on a webpage.
In Selenium, these strategies are accessed through the **By class**.
Before using them, import:

```
from selenium.webdriver.common.by import By
```

---

1. By.ID

Purpose

Locates an element using its **id attribute**.

Why use it?

- usually the **fastest & most reliable**
- best option when the id is unique and stable

Example

```
driver.find_element(By.ID, "username")
```

Explanation

This finds the element whose id="username".

---

2. By.NAME

Purpose

Locates an element using its **name attribute**.

Where it is commonly used

- form fields
- input elements
- radio buttons
- text boxes

Example

```
driver.find_element(By.NAME, "password")
```

Explanation

This finds the element whose name="password".

---

3. By.CLASS_NAME

Purpose

Locates an element using its **class attribute**.

Example

```
driver.find_element(By.CLASS_NAME, "submit-button")
driver.find_elements(By.CLASS_NAME, "list-item")
```

Use carefully

class values are often shared by many elements, so this locator may return multiple matches.

---

4. By.TAG_NAME

Purpose

Locates elements using the HTML tag name.

Common tag names

- a, input, div, h1, button

Example

```
driver.find_elements(By.TAG_NAME, "a")  # Find all links
driver.find_element(By.TAG_NAME, "h1")  # Find the first H1 heading
```

Explanation

- By.TAG_NAME, "a" finds all anchor tags (links)
- By.TAG_NAME, "h1" finds the first <h1> element

Important Note

This locator usually returns many elements, so it is mostly used with find_elements().

---

5. By.LINK_TEXT

Purpose

Locates an <a> element using the **exact visible text** of the link.

Important Note

- works only for links
- text must match exactly
- it is **case-sensitive**

Example

```
driver.find_element(By.LINK_TEXT, "Logout")
```

Explanation

This finds the link whose visible text or inner text is exactly **Logout**.

---

6. By.PARTIAL_LINK_TEXT

Purpose

Locates an <a> element using **part of the visible text**.

Important Note

- works only for links
- partial match is enough
- it is **case-sensitive**

Example

```
driver.find_element(By.PARTIAL_LINK_TEXT, "Forgotten")
```

Explanation

This can match a link with text like:

Forgotten Password?

because only part of the visible text is used.

Example Import

```
from selenium.webdriver.common.by import By
```

Example Usage

```
driver.find_element(By.ID, "username")
driver.find_element(By.NAME, "password")
driver.find_element(By.CLASS_NAME, "submit-button")
driver.find_elements(By.TAG_NAME, "a")
driver.find_element(By.LINK_TEXT, "Logout")
driver.find_element(By.PARTIAL_LINK_TEXT, "Forgotten")
```

If you are **not using By**, older style strings like 'xpath', 'id', 'name' were used in older Selenium versions, but that style is **not recommended now**.

```
driver.find_element("locator_type", "locator_value")
```

Example Usage

```
driver.find_element("id", "username")
driver.find_element("name", "password")
driver.find_element("class name", "submit-btn")
driver.find_element("tag name", "input")
driver.find_element("link text", "Login")
driver.find_element("partial link text", "Forgot")
driver.find_element("xpath", "//input[@id='username']")
driver.find_element("css selector", "input[name='username']")
```

Disadvantages of Old Syntax

1. **Less readable**
   Raw strings like "id" and "xpath" are not as clean as By.ID and By.XPATH.
2. **Easy to make spelling mistakes**
   Example:

driver.find_element("xpah", "//input[@id='username']")

Small typing mistakes are easy to make.

3. **Need to remember exact string format**
   You must type locator names exactly like:
   - "class name"
   - "partial link text"
   - "css selector"
4. **Not consistent in code**
   Old syntax looks outdated and may not match modern Selenium code.
5. **Harder to maintain**
   In large frameworks, string-based locators are less organized and harder to manage.

Advanced Locators – CSS Selectors

1. What are CSS Selectors?

CSS Selectors are patterns used to find HTML elements on a webpage.
In Selenium, we use them to locate elements for automation.
Example:

```
driver.find_element(By.CSS_SELECTOR, "input")
```

---

2. Why use CSS Selectors?

CSS Selectors are useful because they are:

- **Fast** – usually faster than XPath
- **Readable** – shorter and easier to understand
- **Widely supported** – browsers already support CSS

---

3. Basic Syntax

General format:

```
driver.find_element(By.CSS_SELECTOR, "selector")
```

A. Tag Name

Select by HTML tag.
Example:

```
driver.find_element(By.CSS_SELECTOR, "input")
driver.find_elements(By.CSS_SELECTOR, "a")
```

B. ID Selector

Use # before id value.
Example:

```
driver.find_element(By.CSS_SELECTOR, "#username")
```

Equivalent to:

```
driver.find_element(By.ID, "username")
```

C. Class Selector

Use . before class name
Example:

```
driver.find_element(By.CSS_SELECTOR, ".button")
```

Equivalent to:

```
driver.find_element(By.CLASS_NAME, "button")
```

D. Tag + ID
Combine tag and id.
Example:
driver.find_element(By.CSS_SELECTOR, "input#username")
Meaning:

- find an input tag
- with id username

E. Tag + Class
Combine tag and class.
Example:
driver.find_element(By.CSS_SELECTOR, "button.submit")
Meaning:

- find a button
- with class submit

---

4. Attribute Selectors
A. Single Attribute
Syntax:
tagname[attribute='value']
Examples:
driver.find_element(By.CSS_SELECTOR, "input[name='username']")
driver.find_element(By.CSS_SELECTOR, "input[type='password']")
driver.find_element(By.CSS_SELECTOR, "button[type='submit']")

B. Multiple Attributes
Example:
driver.find_element(By.CSS_SELECTOR, "input[type='text'][name='username']")
Meaning:

- input tag
- type = text
- name = username

---

5. Substring Matching
A. Starts With ^=
Example:
driver.find_element(By.CSS_SELECTOR, "input[id^='user']")
Matches ids like:

- user1
- username

B. Ends With $=
Example:
driver.find_element(By.CSS_SELECTOR, "img[src$='.png']")
Matches image source ending with .png

C. Contains *=
Example:
driver.find_element(By.CSS_SELECTOR, "a[class*='button']")
Matches class values containing "button"
Useful for dynamic classes.

---

6. Relationship Selectors
A. Descendant Selector (space)
Syntax:
ancestor descendant
Example:
driver.find_element(By.CSS_SELECTOR, "div#main p")
Meaning:
find any <p> inside div with id main

B. Direct Child Selector >
Syntax:
parent > child
Example:
driver.find_element(By.CSS_SELECTOR, "ul > li")
Meaning:
find only li elements that are direct children of ul
Meaning:
find all <p> elements after <h2> with same parent

Note:
CSS is good for:

- id
- class
- attributes
- parent to child

CSS cannot easily go upward in DOM
That is where XPath is stronger.
Example:

- CSS can go from parent to child
- XPath can go from child to parent too

Advanced Locators – XPath

1. What is XPath?

XPath stands for **XML Path**

It is used to locate elements in a webpage by navigating the HTML structure.

Since HTML is like a tree of elements, XPath helps us move through that tree and select what we want.

Example:

```
driver.find_element(By.XPATH, "//input")
```

---

2. Why use XPath?

XPath is useful because it is very powerful.

Advantages

- Can move **up, down, and sideways** in the DOM
- Can locate elements using **text**
- Good when there is **no id or class**
- Useful for **complex relationships**

Drawbacks

- Can be **slower** than CSS
- Can become **long and difficult**
- **Absolute XPath** breaks easily if page structure changes

---

3. XPath Syntax Basics

General format:

```
driver.find_element(By.XPATH, "xpath_expression")
```

---

4. Path Symbols in XPath

A. Single Slash /

Used for **direct child**

Example:

/html/body/div/input

Meaning:

- go from html
- to body
- to div
- to input

B. Double Slash //

Used to find element **anywhere in the page**

Example:

//input

Meaning:

find all input elements anywhere

---

5. Absolute XPath vs Relative XPath

Absolute XPath

Starts from root of the page.

Example:

/html/body/div[1]/form/input

Problem:

- very long
- breaks easily
- not recommended

Relative XPath

Starts from anywhere using //

Example:

//input[@id='username']

Why better:

- shorter
- easier to read
- more stable

Always prefer **Relative XPath**

---

6. Locate by Attribute

Syntax:

```
//tagname[@attribute='value']
```

Examples:

```
driver.find_element(By.XPATH, "//input[@id='username']")
driver.find_element(By.XPATH, "//input[@name='password']")
driver.find_element(By.XPATH, "//button[@type='submit']")
driver.find_element(By.XPATH, "//img[@alt='Company Logo']")
```

---

7. Locate by Text

A. Exact Text using text()

Example:

```
driver.find_element(By.XPATH, "//button[text()='Login']")
driver.find_element(By.XPATH, "//a[text()='Logout']")
```

This matches exact visible text.

B. Partial Text using contains(text(), ...)

Example:

```
driver.find_element(By.XPATH, "//h2[contains(text(),'Welcome')]")
```

This is useful when text is long or partially dynamic.

---

8. Partial Match Functions

A. starts-with()

Checks whether value starts with something.

Example:

```
driver.find_element(By.XPATH, "//button[starts-with(@id,'btn_')]")
```

Matches:

- btn_login
- btn_submit

B. ends-with()

Example:

```
//img[ends-with(@src,'.png')]
```

Note:

ends-with() is XPath 2.0 and may not work in all browsers.

XPath Axes

Axes are used to move around related elements.

A. parent::

Find immediate parent.

Example:

```
driver.find_element(By.XPATH, "//input[@id='username']/parent::div")
```

B. child::

Find direct child

Example:

```
driver.find_element(By.XPATH, "//div[@id='form']/child::input")
```

C. ancestor::
Find parent, grandparent, or higher
Example:
```
driver.find_element(By.XPATH, "//td[text()='Jason']/ancestor::tr")
```

D. descendant::
Find children, grandchildren, etc.
Example:
```
driver.find_element(By.XPATH, "//table/descendant::td")
```

E. following-sibling::
Find next siblings after current element
Example:
```
driver.find_element(By.XPATH, "//td[text()='Jason']/following-sibling::td[1]")
```

F. preceding-sibling::
Find previous siblings before current element
Example:
```
driver.find_element(By.XPATH, "//td[text()='$51.00']/preceding-sibling::td[1]")
```

Indexing in XPath
A. Select nth element
XPath index starts from **1**, not 0.
Example:
```
driver.find_element(By.XPATH, "//table/tbody/tr[3]")
```
Finds 3rd row.

B. Grouping with parentheses ()
Example:
```
driver.find_element(By.XPATH, "(//input[@type='checkbox'])[1]")
```
Finds first checkbox from all matching checkboxes.

Note: You can't access 'svg' tag elements using XPATH but you can access it using CSS Selectors

Working with Web Elements

- **What is a WebElement?**

When we use driver.find_element(), Selenium returns a WebElement object.
This object allows us to:

- Perform actions (click, type, etc.)
- Retrieve information (text, attributes, state)

Common Actions on Web Elements

- **element.click()**
  - Simulates a **mouse click**
  - Used for buttons, links, checkboxes, etc.

```
button.click()
```

- **element.send_keys()**
  - Used to type text into input fields

```
element.send_keys("Hello")
```

Multiple inputs:

```
element.send_keys("Hello", " World")
```

With keyboard keys:

```
from selenium.webdriver.common.keys import Keys
element.send_keys("Hello", Keys.ENTER)
```

- **element.clear()**
  - Clears existing text before typing

```
element.clear()
```

Getting Information from Elements

- **element.text**
  - Gets **visible text**

```
print(element.text)
```

- **element.get_attribute()**
  - Gets value of any HTML attribute

```
element.get_attribute("value")
element.get_attribute("href")
element.get_attribute("class")
```

- **element.is_displayed()**
  - Checks if element is **visible** ; gives Boolean output

```
element.is_displayed()
```

- **element.is_enabled()**
  - Checks if element is **enabled**

```
element.is_enabled()
```

- **element.is_selected()**
  - For checkboxes/radio buttons

```
element.is_selected()
```

- **driver.current_url**
  - Gets current page URL

```
print(driver.current_url)
```

- **driver.title**
  - Gets page title

```
print(driver.title)
```

Synchronization Issues & Waits in Selenium

- Synchronization means aligning the speed of Selenium script with the web application.
- Selenium runs faster than the browser, but web elements may load slowly.
- Because of this:
  - Element may **not be present**
  - Element may **not be visible**
  - Element may **not be clickable**

Implicit Wait

Implicit Wait is a global synchronization mechanism in Selenium where the WebDriver waits for a specified amount of time while trying to find elements before throwing an exception.

Implicit wait tells Selenium to keep checking for an element for a certain time before failing.

Key Behaviour:

- Applied globally to all find_element() calls
- Works only for:
  - find_element
  - find_elements
- Once set → remains active for entire session

How It Works Internally

1. driver.find_element(...) is called.
2. If the element is found immediately, execution continues.
3. If not found, WebDriver waits, repeatedly checking for the element's presence in the DOM every ~500ms (polling frequency is internal) until the timeout (e.g., 10 seconds) is reached.
4. If found within the timeout, the element is returned.
5. If not found after the timeout, NoSuchElementException is raised (or an empty list for find_elements).

Syntax

driver.implicitly_wait(10) = wait for 10 seconds

Example

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.get("https://example.com")

driver.implicitly_wait(10)

element = driver.find_element(By.ID, "username")
element.send_keys("Nikhil")
```

Advantages

- Easy to implement
- Reduces NoSuchElementException
- Good for simple/static pages
- Global configuration

Disadvantages

- Not flexible
- Cannot wait for conditions (clickable, visible, etc.
- Slows down test if element not present

Explicit Waits

Implicit wait only checks for the presence of an element in the DOM. It doesn't guarantee the element is visible, clickable, or that some other condition (like text appearing) has been met. For dynamic applications where elements appear, disappear, enable/disable, or update.

A mechanism where you tell WebDriver to wait for a specific condition to be true for a specific element (or general condition like an alert) before proceeding, up to a maximum timeout period. If the condition is met within the timeout, the script continues immediately. If not, a TimeoutException is thrown.

Core Components:

>WebDriverWait: The primary class for implementing explicit waits. You create an instance of it, providing the driver and a timeout in seconds.
>expected_conditions (commonly imported as EC): A module containing a set of predefined conditions to wait for.
>wait.until() is the heart of explicit wait in Selenium.

Internal Working:

- Selenium repeatedly calls the given Expected Condition (EC)
- Default polling interval = 500 milliseconds (0.5 sec)
- It keeps checking until:

Condition returns a truthy value
→ (like a WebElement or True)
Or timeout is reached
→ Throws TimeoutException

Syntax :

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

wait = WebDriverWait(driver, 10)
element = wait.until(EC.visibility_of_element_located((By.ID, "username")))
```

Common Conditional Wait Conditions

1. Element is Visible

EC.visibility_of_element_located(locator)

Element is present + visible on UI

2. Element is Clickable

EC.element_to_be_clickable(locator)

Visible + Enabled

3. Presence of Element

EC.presence_of_element_located(locator)

Exists in DOM (may not be visible)

4. Alert is Present

EC.alert_is_present()

Waits until alert is present

5. Frame Available & Switch

EC.frame_to_be_available_and_switch_to_it(locator)

Waits and switches to an iframe

6. URL Contains

EC.url_contains("partial_url ")

Checks if the current URL contains the given string

EC.url_to_be("expected_url")

Checks for an exact URL match.

7.Windows

EC.number_of_windows_to_be(expected_number)

Checks if the number of open browser windows matches the expected number.

8.Title

EC.title_is("Expected Title")

Checks for an exact title match.

EC.title_contains("Partial Title")

Checks for a partial title match.

Eg:

```
wait = WebDriverWait(driver, 10)

# Wait until login button is clickable
login_btn = wait.until(EC.element_to_be_clickable((By.ID, "login")))
login_btn.click()
```

When to Use Which Condition?

Scenario	Best Condition
Element just loaded	presence_of_element_located
Element visible on UI	visibility_of_element_located
Clicking button	element_to_be_clickable
After page refresh	staleness_of
Waiting for message	text_to_be_present_in_element

Advantages

- More efficient than sleep()
- Reduces test flakiness
- Improves execution speed
- Handles dynamic content properly

Disadvantages

- Wrong condition → still fail
- Slightly complex to implement
- Requires understanding of conditions

Note:

Do NOT mix Implicit Wait and Explicit Wait excessively

Why?

- Can cause **unexpected delays**
- Timeout becomes unpredictable

Fluent Wait

Fluent Wait in Python Selenium is a type of explicit wait implemented using WebDriverWait, where you can define the maximum timeout, customize the polling frequency, and ignore specific exceptions while waiting for a condition to be met.

```
wait = WebDriverWait(driver, timeout=10, poll_frequency=2)
 # check every 2 seconds

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
driver.get("https://example.com")
```

```
wait = WebDriverWait(driver, 10, poll_frequency=1)

# Wait until element is visible
element = wait.until(EC.visibility_of_element_located((By.ID, "username")))

element.send_keys("Hello")
```

Difference between Implicit and Explicit wait

Implicit Wait	**Explicit Wait**
Global wait applied to all elements	Applied to specific element/condition
Set once for entire WebDriver session	Used wherever required in the code
Waits for element presence only	Waits for specific conditions (visibility, clickable, etc.)
Not condition-based	Condition-based
Less flexible	Highly flexible
Applies to find_element and find_elements only	Works with multiple conditions using ExpectedConditions on that until
May slow execution if element not found	Faster, stops when condition is met
Simple to implement	Slightly more complex

Action Chains in Selenium

Action Chains in Selenium allow you to perform complex user interactions that cannot be done with simple commands.

These simulate **real user behavior** like:

- Mouse hover
- Right click
- Double click
- Drag and drop
- Keyboard actions

Why Do We Need Action Chains?

Normal Selenium methods like:

element.click() may fail when:

- Element is hidden until hover
- Needs drag-drop interaction
- Requires keyboard + mouse combination

Action Chains solve these by mimicking actual user gestures

Import and Setup

```
from selenium import webdriver
from selenium.webdriver.common.action_chains import ActionChains

driver = webdriver.Chrome()
actions = ActionChains(driver)
```

Common Mouse Actions

- **Mouse Hover (Move to Element)**

Used for dropdown menus

```
menu = driver.find_element("xpath", "//a[@id='menu']")
actions.move_to_element(menu).perform()
```

- Flipkart / Myntra category hover

- **Click**

```
element = driver.find_element("id", "btn")
actions.click(element).perform()
```

- **Double Click**

```
element = driver.find_element("id", "double")
actions.double_click(element).perform()
```

- **Context Click(Right Click)**

```
element = driver.find_element("id", "right")
actions.context_click(element).perform()
```

- **Drag and Drop**

```
source = driver.find_element("id", "drag")
target = driver.find_element("id", "drop")

actions.drag_and_drop(source, target).perform()
```

Keyboard Actions

Import Keys:

```
from selenium.webdriver.common.keys import Keys
```

- **Page Up & Page Down**

```
actions.send_keys(Keys.PAGE_DOWN).perform()
```

scrolls little bit down

```
actions.send_keys(Keys.PAGE_UP).perform()
```

scrolls little bit up

- **Key Down + Key Up**

```
actions.key_down(Keys.SHIFT).send_keys("hello").key_up(Keys.SHIFT).perform()
```

Output → HELLO

- **Copy-Paste Example**

```
actions.key_down(Keys.CONTROL).send_keys("a").key_up(Keys.CONTROL).perform()
actions.key_down(Keys.CONTROL).send_keys("c").key_up(Keys.CONTROL).perform()
actions.key_down(Keys.CONTROL).send_keys("v").key_up(Keys.CONTROL).perform()
```

- **Pause**

```
actions.pause(2)
```

Handling Dropdown

- Standard HTML dropdowns are created using the <select> tag, containing multiple <option> tags.
- Selenium provides a dedicated Select class to make interacting with these easier than just clicking.
- In Selenium, dropdowns are handled using a dedicated class:

Select class

Easier than using normal click()

Provides built-in methods for selection

Import Statement

```
from selenium.webdriver.support.select  import Select
```

Step 1: Locate the <select> element

```
from selenium.webdriver.common.by import By
dropdown_element = driver.find_element(By.ID, "dropdown")
```

Step 2: Create Select Object

```
select_object = Select(dropdown_element)
```

Step 3: Select Options

1. Select by Visible Text (Most Common)

```
select_object.select_by_visible_text("Option Text Here")
```

Matches exact visible text

Most readable and widely used

2. Select by Value

```
select_object.select_by_value("value_attribute_content")
```

Uses value attribute

Useful when UI text changes but value remains constant

3. Select by Index

```
select_object.select_by_index(index_number)
```

Uses position (0-based index)

Least reliable (index may change)

Getting Information from Dropdown

1. Get All Options

```
options = select_object.options
for option in options:
    print(option.text)
```

Returns list of all <option> elements

2. Get All Selected Options

```
selected = select_object.all_selected_options
```

Returns list of selected options

Single-select → 1 element

Multi-select → multiple elements

3. Get First Selected Option

```
first = select_object.first_selected_option
print(first.text)
```

Useful for single-select dropdowns

Handling Multi-Select Dropdowns (<select multiple>)

- Same Select class is used
- You can select multiple options

```
select_object.select_by_visible_text("Option 1")
select_object.select_by_visible_text("Option 2")
```

Deselect Methods (Only for Multi-Select)

```
select_object.deselect_by_visible_text("Option 1")
select_object.deselect_by_value("value")
select_object.deselect_by_index(2)
select_object.deselect_all()
```

`deselect_all()` → removes all selections

Check if Dropdown is Multi-Select

```
select_object.is_multiple()
```

Returns:

- True → Multi-select
- False → Single-select

Note: Not all dropdowns use <select> tag

Some modern dropdowns are built using:

-
-
- <ul>

In such cases, **Select class will NOT work**

You must use:

```
element.click()
```

And handle using:

- Locators
- Click actions

Handling Multiple Windows / Tabs

Scenario

- Clicking a link/button may open a **new browser window or tab**
- Selenium **does NOT automatically switch** to it
- It stays on the original window

Window Handles

- Each window/tab has a unique **window handle (ID)**
- Example: {'handle1', 'handle2'}

Steps

Step 1: Store Original Window

```
original_window_handle = driver.current_window_handle
```

Step 2: Perform Action

```
element.click()
```

Step 3: Get All Window Handles

```
all_handles = driver.window_handles
```

Step 4: Switch to New Window

```
  driver.switch_to.window(new_window_handle)
  print(f"Switched to new window with title: driver.title")
```

Step 5: Perform Actions in New Window

Step 6: Switch Back to Original

```
driver.switch_to.window(original_window_handle)
```

Selenium 4 Improvement

```
# Open new tab
driver.switch_to.new_window('tab')
driver.get("https://newsite.com")

# Open new window
driver.switch_to.new_window('window')
driver.get("https://anothersite.com")
```

Note:

- Works for opening manually
- Not useful when click opens new window

Handling Frames / iFrames

Scenario

- Some pages use: <iframe>
- These contain **separate HTML documents** (like nested html)

Problem

- Selenium can only see **current DOM**
- Cannot access elements inside frame directly

Solution

- Switch into the frame before interacting

Switching Methods

```
iframe_element = driver.find_element(locator)
driver.switch_to.frame(iframe_element)
```

Switching Back

- **To Main Page**

```
driver.switch_to.default_content()
```

- **To Parent Frame**

```
driver.switch_to.parent_frame()
```

Using explicit wait: it'll find the iframe then switch to it

```
wait.until(EC.frame_to_be_available_and_switch_to_it(iframe_locator))
print("Switched into the iframe.")
```

Handling JavaScript Alerts / Confirmations / Prompts

Scenario

- Popups triggered by actions (delete, confirm, etc.)
- Types:
  - Alert
  - Confirmation
  - Prompt

Problem: Blocks interaction with main page

Solution: Switch to alert

Switching to Alert

```
wait = WebDriverWait(driver, 5)
alert = wait.until(EC.alert_is_present())
```

OR (less reliable):

```
alert = driver.switch_to.alert
```

Alert Actions

Method	Description
alert.text	Get alert message
alert.accept()	Click OK
alert.dismiss()	Click Cancel
alert.send_keys()	Enter text (only prompt)

- **Simple Alert**

```
driver.find_element(By.XPATH, "//button[text()='Click for JS Alert']").click()
alert = wait.until(EC.alert_is_present())
alert_text = alert.text
print(f"Simple Alert text: '{alert_text}'")
alert.accept()
print("Accepted simple alert.")
result = driver.find_element(By.ID, "result").text
print(f"Result after alert: '{result}'")
```

- **Confirmation Alert**

```
driver.find_element(By.XPATH, "//button[text()='Click for JS Confirm']").click()
confirm = wait.until(EC.alert_is_present())
confirm_text = confirm.text
print(f"Confirmation Alert text: '{confirm_text}'")
confirm.dismiss() # Click Cancel
print("Dismissed confirmation alert.")
result = driver.find_element(By.ID, "result").text
print(f"Result after confirmation: '{result}'")
```

- **Prompt Alert**

```
driver.find_element(By.XPATH, "//button[text()='Click for JS Prompt']").click()
prompt = wait.until(EC.alert_is_present())
prompt_text = prompt.text
print(f"Prompt Alert text: '{prompt_text}'")
prompt.send_keys("Hello Prompt!") # Enter text
print("Entered text into prompt.")
prompt.accept() # Click OK
print("Accepted prompt.")
result = driver.find_element(By.ID, "result").text
print(f"Result after prompt: '{result}'")
assert "Hello Prompt!" in result
```

❖ Scrolling using JavaScriptExecutor

Problem

- Elements may be present in DOM but **not visible in viewport**
- Selenium tries auto-scroll, but:
  - May fail sometimes
  - Cannot handle lazy loading properly

Solution

Use **JavaScript execution** via Selenium, allows executing custom JavaScript inside browser

Usage

```
driver.execute_script("JavaScript Code")
```

➢ **Common Scrolling Scripts**

Scroll to Bottom

```
driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")
```

Scroll to Top

```
driver.execute_script("window.scrollTo(0, 0);")
```

Scroll Element into View

```
element_to_scroll_to = driver.find_element(By.ID, "myElement")
driver.execute_script("arguments[0].scrollIntoView(true);",element_to_scroll_to)
```

Scroll by Specific Pixels

```
#Scroll down 500px
driver.execute_script("window.scrollBy(0, 500);")
# Scroll up 200px
driver.execute_script("window.scrollBy(0, -200);")
```

❖ Screenshots

Purpose

- Debugging failures
- Visual verification
- Reporting

Methods

Full Page Screenshot

`driver.save_screenshot("path/to/filename.png")` : Captures entire visible browser viewport

Element Screenshotelement.screenshot("path/to/element.png") : Captures only specific element

Example:

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.get("https://www.google.com")

# Take screenshot
driver.save_screenshot("full_page.png")
search_box = driver.find_element(By.NAME, "q")

# Take screenshot of element
search_box.screenshot("search_box.png")

driver.quit()
```

❖ File Upload

```
file_input = driver.find_element(By.CSS_SELECTOR, "input[type='file']")

file_path = r"C:\\path\\to\\your\\local\\file\\upload.txt"

file_input.send_keys(file_path)
```

❖ File Download

Clicking button triggers file download